ReLU and Its Improvements in Machine Learning

Name: Dnyaneshwari Maradkar

Student No: 24142497

GitHub Repository: https://github.com/dnyaneshwari7/machine-learning-relu-improvements

1. Introduction

When I first started learning about neural networks, one thing that seemed confusing was how a model made up of mathematical operations could learn something as complex as recognising objects, understanding patterns, or making useful predictions from data. What makes this possible is the activation function. Activation functions introduce non-linearity into a neural network, which means the network can move beyond simple straight-line relationships and learn more realistic patterns from data. Without an activation function, even a deep network with many layers would behave like a basic linear model.

Among the many activation functions used in machine learning, the Rectified Linear Unit, usually called ReLU, is one of the most widely used. It became popular because it is easy to implement, computationally efficient, and effective in practice. In many deep-learning models, ReLU trains faster and more reliably than older activation functions such as sigmoid and tanh. However, even though ReLU is useful, it is not perfect. It has a well-known weakness called the dying ReLU problem, where some neurons can stop learning completely. Because of this limitation, improved versions of ReLU were developed.

This report explains the ReLU activation function, why it is useful, where it falls short, and how improved activation functions such as Leaky ReLU, PReLU, ELU, GELU, and Swish attempt to solve its weaknesses. The goal is not only to describe the formulas, but also to explain in simple terms why small mathematical changes can make a big difference to how a neural network learns.

2. The ReLU Activation Function

The ReLU activation function is defined as follows:

$$f(x) = max(0, x)$$

This can also be written in piecewise form:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

This means that ReLU keeps any positive input unchanged, but turns every negative input into zero. The reason this function is so popular is that it is very simple compared with older activation functions. For

example, sigmoid and tanh require exponential calculations, while ReLU only needs to compare the input with zero. That makes it faster to compute, especially when the activation function is being applied millions of times during training.

The derivative of ReLU is:

$$f(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

At $x = 0$, the derivative is not strictly defined, but in practical implementations this is usually handled without difficulty. The important point is that for positive inputs the gradient remains equal to 1. This helps information and gradients pass through the network during backpropagation, which is one of the main reasons ReLU became the default activation function in many deep-learning models.

3. Why ReLU Works So Well

ReLU is useful for several practical reasons. First, it is computationally efficient. Since the function only compares the input with zero, it is significantly simpler than sigmoid or tanh. This means the training process can be faster, especially for large networks and large datasets.

Second, ReLU helps reduce the vanishing gradient problem. With activation functions such as sigmoid, gradients can become extremely small as they pass through many layers, which makes it hard for the earlier layers in a deep network to learn. ReLU avoids this issue in the positive region because its gradient does not shrink. When the input is positive, the derivative stays at 1, so learning can continue more effectively.

Third, ReLU often creates sparse activations. Because all negative inputs become zero, some neurons remain inactive for a particular example. This can make the network more efficient and, in some cases, may help reduce overfitting. In the positive region, ReLU behaves like a straight line, which means it preserves useful information instead of compressing it into a narrow range. This is one of the reasons it performs well in modern neural networks.

4. Limitations of ReLU

4.1 The Dying ReLU Problem

The biggest weakness of ReLU is the dying ReLU problem. Since the function outputs zero for all negative inputs, the gradient in that region is also zero. If a neuron keeps receiving negative inputs during training, its weights may stop updating because there is no gradient signal to push learning forward. When this happens, the neuron can effectively become inactive, or "dead."

A few dead neurons may not destroy the entire network, but if many neurons stop learning, the model loses some of its ability to represent complex patterns. In other words, the network still works, but not as well as it could. This is why researchers started looking for activation functions that keep the advantages of ReLU while avoiding a completely flat negative region.

4.2 Other Limitations

ReLU also has a few other limitations. Its outputs are not zero-centred, since they are always zero or positive. This can make optimisation slightly less balanced. In addition, ReLU is unbounded on the positive side, which means large inputs can produce very large outputs. Finally, ReLU has a sharp corner at zero, so it is not fully smooth. Although this is usually manageable in practice, smoother activation functions can sometimes improve optimisation.

5. Improved Activation Functions

Because ReLU is powerful but imperfect, several alternative activation functions were developed. These improved functions mainly focus on changing the negative region so that neurons can continue learning even when the input is below zero.

5.1 Leaky ReLU

Leaky ReLU keeps the positive side the same as ReLU, but gives the negative side a small slope instead of making it completely flat:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

Here, α is a small constant, often 0.01. This means negative values are not completely removed. Instead, they produce a small negative output. As a result, the gradient in the negative region is small rather than zero, which reduces the chance of neurons dying.

5.2 Parametric ReLU (PReLU)

PReLU is similar to Leaky ReLU, but instead of fixing the negative slope in advance, the model learns it during training:

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

In this case, a is a learnable parameter. This makes PReLU more flexible because the network can adapt the negative slope based on the data. In some problems, this can lead to better performance than standard ReLU or Leaky ReLU.

5.3 Exponential Linear Unit (ELU)

ELU introduces a smooth exponential curve for negative inputs:

$$f(x) = \begin{cases} -x, & x > 0 \\ \alpha(e^{x-1}), & x \leq 0 \end{cases}$$

Unlike ReLU, ELU produces negative outputs for negative inputs instead of zero. This helps push average activations closer to zero, which can improve optimisation. The transition around zero is also smoother than ReLU, which may help the network train more steadily.

5.4 GELU

GELU, or Gaussian Error Linear Unit, is commonly used in transformer-based models. A widely used approximation is:

$$GELU(x) = 0.5x(1 + \tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)))$$

Instead of making a hard decision like ReLU, GELU smoothly weights the input. This means small values are reduced gradually rather than being sharply cut off. Because of this smoother behaviour, GELU is often preferred in large language models and other modern architectures.

5.5 Swish

Swish is another smooth activation function defined by:

$$f(x) = x \cdot \sigma(x)$$

where the sigmoid function is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Swish allows small negative values to pass through and creates a smooth curve across the whole input range. It is more flexible than ReLU, although it is also slightly more expensive to compute. In some deep networks, Swish has been shown to perform better than standard ReLU.

6. Python Implementation and Visualisation

To understand these functions more clearly, I implemented them in Python using NumPy and Matplotlib. The code focuses on three main visual ideas: the positive region of ReLU, the negative region where the dying ReLU problem appears, and a comparison between ReLU and the improved activation functions.

The positive-region graph shows why ReLU is attractive: for positive inputs, the function behaves like a straight line and keeps useful information unchanged. The negative-region graph shows the main

weakness: all negative inputs are mapped to zero, which explains why neurons may stop learning. The comparison graph then makes it easy to see how Leaky ReLU, PReLU, ELU, GELU, and Swish change the negative side to keep some form of gradient or smoother behaviour.

These visualisations are important because they turn the formulas into something easier to interpret. A small change in the shape of the curve can have a major effect on optimisation, gradient flow, and overall model performance.

7. Discussion

One of the most useful lessons from studying activation functions is that there is no single best choice for every problem. ReLU remains a very strong default because it is simple, fast, and effective in many standard neural-network architectures. For many image classification and feedforward tasks, it still performs very well.

However, if a model shows signs of inactive neurons or unstable optimisation, improved functions may be better choices. Leaky ReLU and PReLU are practical when the main concern is preventing dead neurons. ELU is useful when smoother negative behaviour and more balanced activations are preferred. GELU and Swish are often chosen in more advanced deep-learning systems where smoothness and flexibility can provide a performance advantage.

This shows that activation functions are not just small technical details. They directly influence how well a network learns, how quickly it trains, and how stable the optimisation process becomes.

8. Conclusion

ReLU is one of the most important activation functions in machine learning because it is simple, computationally efficient, and highly effective. Its formula, $f(x) = max(0, x)$, is easy to implement, yet it made training deep neural networks much more practical. At the same time, its biggest weakness is the dying ReLU problem, where neurons may stop learning if they remain in the negative region.

Improved activation functions were created to address this issue. Leaky ReLU introduces a small negative slope, PReLU learns that slope automatically, ELU uses an exponential negative region, and GELU and Swish provide smoother alternatives. Together, these functions show that even a small mathematical change can significantly affect the behaviour and performance of a neural network.

Overall, studying ReLU and its variants gives a clearer understanding of why activation functions matter so much in deep learning. They are not just formulas placed between layers; they play a central role in whether a model learns well or struggles during training.

9. References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Nair, V., & Hinton, G. E. (2010). Rectified Linear Units Improve Restricted Boltzmann Machines. Proceedings of ICML 2010.
- Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier Nonlinearities Improve Neural Network Acoustic Models.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV 2015.
- Clevert, D.-A., Unterthiner, T., & Hochreiter, S. (2016). Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs).
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for Activation Functions.